Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 23

Radix Trees

A Data Structure for Storing Strings

- How can we store a set of strings and search in it efficiently?
  - Option we already have: Use std::set<std::string>
    - Comparing a pair of strings takes time proportional to the length of the (shorter) string
    - Searching, inserting, deleting in a set of n strings of length D takes O(D log n) time
  - Option : Use a radix tree
    - Searching, inserting, deleting in a set of n strings of length D takes O(D) time!

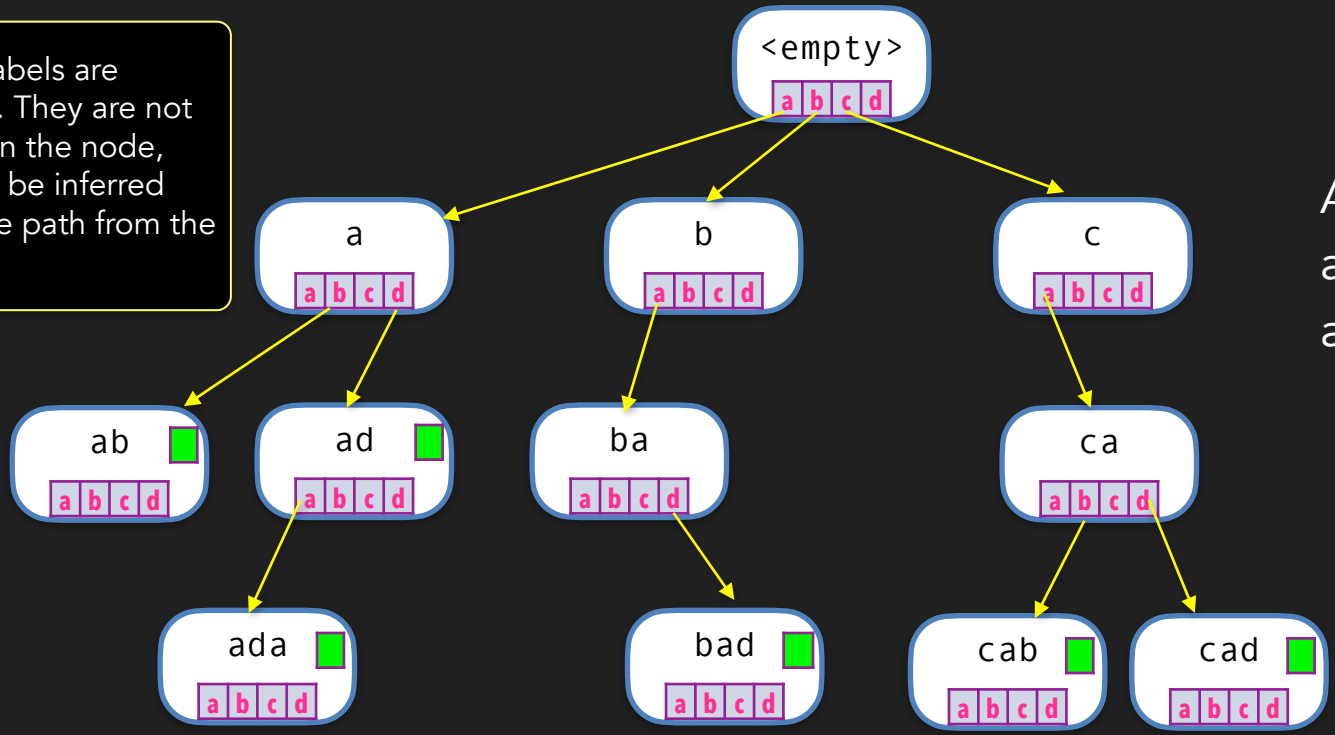
Trie

- A Radix Tree is an optimisation of a simpler data structure called a Trie
- For storing strings over an alphabet of size B:
  - A full B-ary tree: Each internal node has B children, each child labelled by a distinct character from the alphabet
  - The labels on the path from the root to a node is a string that is associated with that node
  - For each prefix of a string that is stored, there is a unique internal node. Leaves correspond to minimal strings which are not such prefixes.
  - Each node has a boolean flag `full_entry` indicating whether the corresponding string is stored, or if it is merely a prefix (alternately/ equivalently, include a special end of string character in the alphabet)

Trie

- Strings: { ab, ad ada, bad, cab, cad }. Alphabet: (a, b, c, d)

Node labels are implicit. They are not stored in the node, but can be inferred from the path from the root.

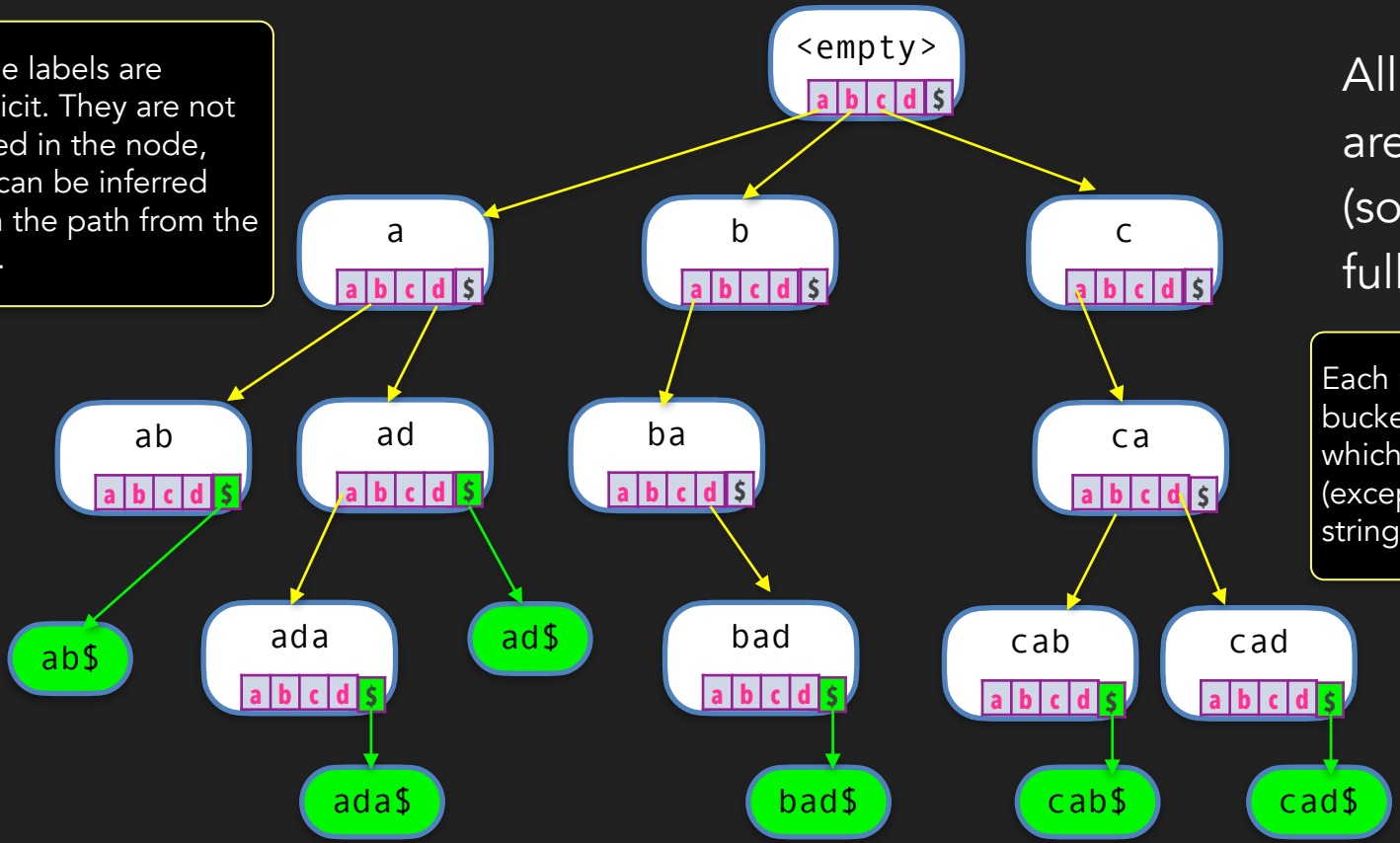


All internal nodes are prefixes. Some are also full entries.

Trie

- Alternate view: If `full_entry` set, has a leaf child that is a full entry

Node labels are implicit. They are not stored in the node, but can be inferred from the path from the root.



All internal nodes are prefixes. Only (some) leaves are full entries.

Each node corresponds to a bucket in MSD Radix Sort which is recursively sorted (except nodes with only one string left)

Trie

- Searching in a trie: Simply follow the unique path determined by the characters of the string ("MSD" first)
  - If the path exists, and further when the string ends, the full_entry flag is set (i.e., child for $ also exists), then the string is present
- Time: O(D) for searching D-long strings
  - Independent of how many strings are stored in the trie!
  - cf. Using std::set<std::string> requires Θ(D log n) time
- Can also search for all strings with a given prefix query, or the strings that share the Longest Common Prefix with a given query string
  - Important in many applications: Internet Packet routing tables, autocomplete from a dictionary (e.g., in IDEs, browser URLs, tab completion in shells, ...)

Trie

- Trie also supports easy insertion and deletion
  - Insertion: Search to find the longest prefix, and insert a chain of nodes from there
  - Deletion: Find the longest prefix which has at least two children, and delete all the nodes in the chain from there till the entry being deleted
  - Handle the full_entry flag correctly when inserting/deleting the leaf in these chains
    - Inserting the leaf node: just set the full_entry flag in the parent
    - Deleting the leaf node: just reset the full_entry flag in the parent

Attendance Break!

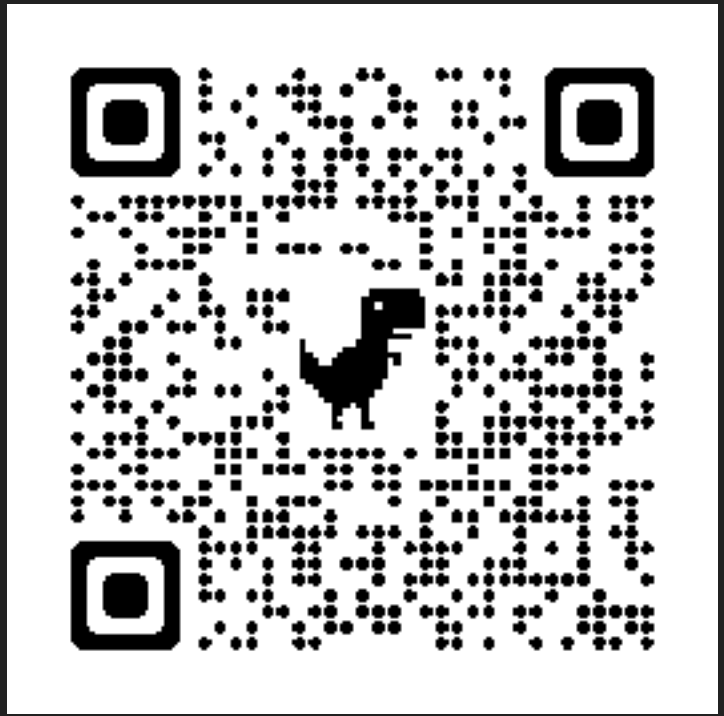
Login using CSE LDAP id at
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

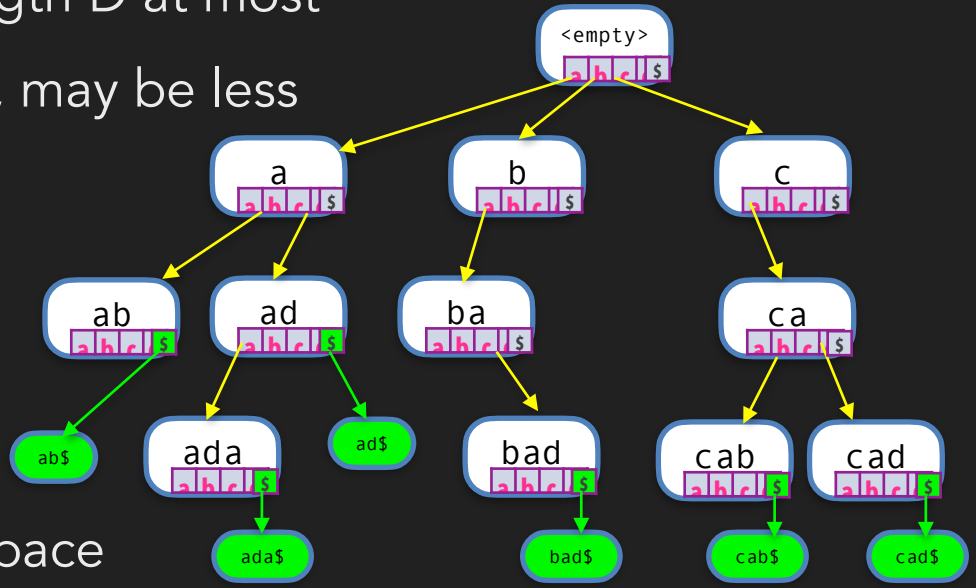
The selected students should come to the front

If you are not in the classroom, a report will be sent to DDAC!



Space in a Trie

- Each internal node takes O(B) space for the B child pointers
- O(nD) nodes for n strings of length D at most
  - By sharing common prefixes, may be less
- Total space: O(nDB)
- Compare to O(nD) for std::set<std::string>
- Can we do better?
  - Yes, two avenues of saving space
    - Don't use Θ(B) space for all nodes
    - Reduce the total number of nodes

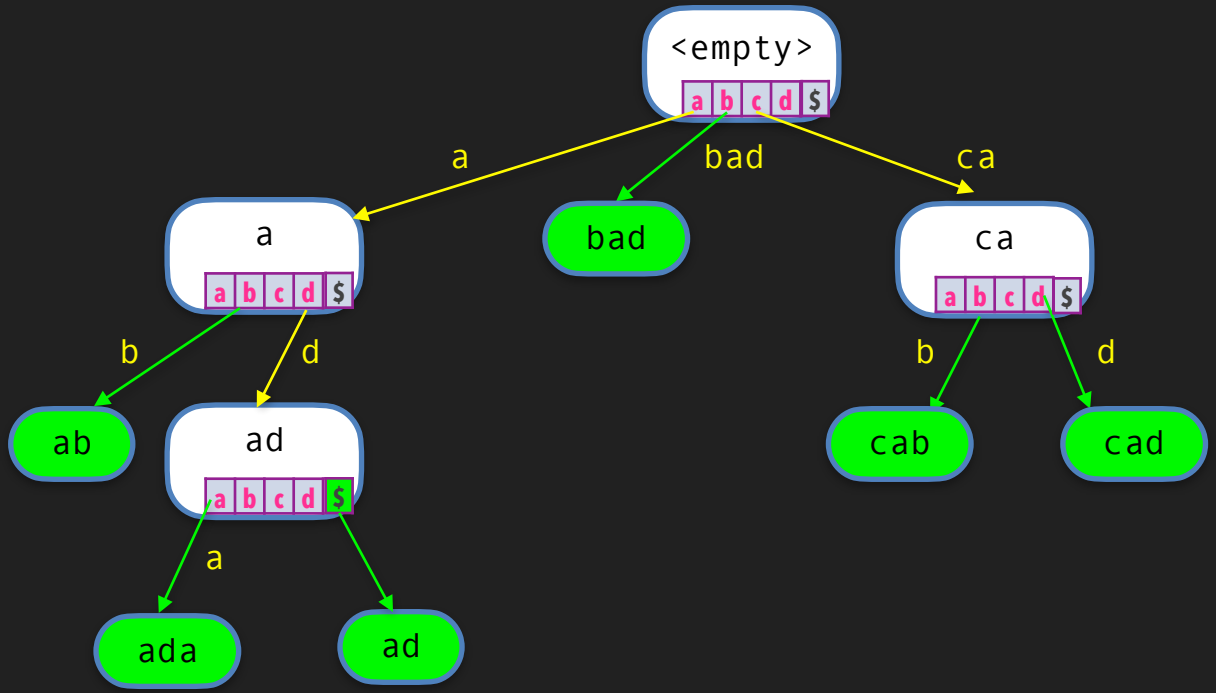


Reducing Space per Node

- Instead of O(B) space, we can use just O(#children) space
  - Use a map (e.g., BST as in std::map or Hash Tables as in std::unordered_map) to store pointer to each child, keyed using the alphabet of size B.
    - std::map (as opposed to std::unordered_map) changes lookup time from O(1) to O(log B), but retains ability to search for smallest string with a prefix, etc.
    - Can adaptively switch the representation, as the number of children changes: when the number of children is small, just store a sorted vector
- How many edges?
  - O(nD) nodes with O(B) children. O(nDB). No gain?
  - Count by the other end of the edge: O(nD) edges!

Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)



Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)
- How many nodes?
  - Recall: if every internal node has exactly 2 children, n leaves ⇒ n-1 internal nodes (proof by induction)
    - Trimming two leaves with the same parent reduces the number of leaves by 1 and number of internal nodes by 1
  - If every internal node has ≥ 2 children, n leaves ⇒ ≤ n-1 internal nodes
    - Trimming k ≥ 2 leaves reduces the number of leaves by k-1 and number of internal nodes by 1

Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)
- How many nodes?
  - n leaves $\Rightarrow \leq$ n-1 internal nodes (except if n=1)
- Total number of nodes is now independent of the string length
- Edge-labels (stored in a map at the parent-node) take a total of O(nD) space
- O(n) space for tree, O(nD) space for edge-labels $\Rightarrow$ O(nD) total space
- O(D) or O(D log B) search time (depending on how the edge map is implemented)

Radix Tree

- Searching in a radix tree requires checking if edge label matches the remainder of the query string
- Inserting a string into a radix tree could require splitting an edge into two (where the mismatch with the inserted string occurs), and inserting a node in between (which then gets a second child corresponding to the inserted string)
- Deleting a string may cause a (non-root) node to have only one child left. Then the node is eliminated and its incoming edge and outgoing edge are merged.

Substring-Matching Using a Suffix Tree

- Goal: To search for query substrings in a long text
- A Radix Tree allows searching for prefixes of a set of strings
- Any substring of T is a prefix of some suffix of T!
- If we store all the suffixes of the text in a Radix Tree, we can query any substring!
  - Suffix Tree of T: A Radix Tree of all the suffixes of T
- There is an O(D) time algorithm ("Ukkonen's algorithm") for constructing a suffix tree for a D-long text
  - Subsequently, searching for a substring of length d takes O(d) time